1. Object Oriented Paradigm:

The Object-Oriented Programming (OOP) paradigm is a way of **writing computer code** by **organizing** it around **objects** and **classes** rather than just **functions** and **step-by-step** logic. It helps model **real-world** things so software is easier to build and fix.

Core Building Blocks

- **Class**: A blueprint or template that defines what data and actions an item should have (like house blueprint).
- **Object**: A real copy made from that class (like the actual built house).
- **Data (Attributes)**: The details or traits of an object (like color or size).
- **Methods (Behaviors)**: The actions an object can perform (like running or stopping).

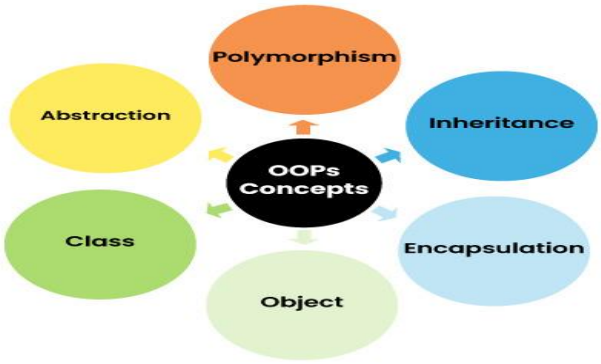
Four Main Principles

- **Encapsulation**: Hiding an object's data inside so other parts of the program cannot change it by mistake.
- **Abstraction**: Showing only the simple parts while hiding the complex inner work (like using a TV remote).
- **Inheritance**: Letting a new child class take properties from an older parent class to reuse code.
- **Polymorphism**: Using the same action name to do different jobs depending on which object are using it.

2. Basic Concepts of OOPS:

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around **data, or objects**, rather than functions and logic. An object is a data field that has unique attributes and behavior.

The system relies on Six Foundational Building Blocks and Core Pillars.



The Two Core Building Blocks

Class: A user-defined blueprint or template used to create individual objects. It defines the structure of data attributes and code functions.

Object: An individual, self-contained instance created from a class template. It represents real-world entities like a specific user, car, or bank account.

The Four Pillars of OOP

Encapsulation:

Encapsulation bundles data (attributes) and the methods (functions) that manipulate that data into a **single unit or class**. It restricts direct access to some of an object's components, which prevents accidental data modification.

→**How it works**: Data is hidden using private access modifiers. It can only be read or changed via public methods like "getters" and "setters".

→**Real-world analogy**: A capsule pill hides its internal chemical mix from the outside, delivering the medicine safely through its outer shell.

Abstraction:

Abstraction **hides complex background implementation details** and exposes only the **essential features** to the **user**. It helps to reduce programming complexity and allows developers to change internal implementation details without impacting external users.

- **How it works**: Developers create simple interfaces or abstract classes to interact with complex logic.
- **Real-world analogy**: When you drive a car, you push the gas pedal to speed up. You do not need to understand how the engine burns fuel or manages the transmission to use it.

Inheritance:

Inheritance is a mechanism that allows a new class (child class) to **derive properties and behaviors** from an existing class (parent class). This promotes code reusability and establishes a logical hierarchy.

- **How it works**: The child class automatically acquires the methods and variables of the parent class, but can also define its own unique attributes.
- **Real-world analogy**: A "Car" class and a "Truck" class can both inherit common traits like "wheels" and "engine" from a base "Vehicle" parent class.

Polymorphism:

Polymorphism translates to "many forms". It allows different classes to be treated as instances of the same parent class, enabling a single interface to **execute different actions depending on the object** invoking it.

- **How it works**: It is achieved via method overloading (defining multiple methods with the same name but different parameters) and method overriding (redefining a parent method inside a child class).
- **Real-world analogy**: Think of a single "Play" command. If you issue it to an audio player, it streams music; if you issue it to a theatre actor, they begin a stage performance.

3. Benefits of OOPS:



Modularity:

Modularity in software engineering is the practice of breaking a large system down into smaller, independent pieces called modules. Key concepts include high cohesion, low coupling, and clear interfaces

Core Principles

Cohesion: A module should do one specific job and do it well.
Coupling: Modules should have minimal dependencies on each other.
Abstraction & Information Hiding: Internal code details stay hidden behind a clean public interface

Benefits of Modular Design

Maintainability: Easier to locate and fix bugs in small chunks of code.
Reusability: Code written for one module can be used again in other projects.
Parallel Work: Multiple team members or groups can build separate modules at the same time.
Easier Testing: Units can be tested individually as black boxes.

Reusability:

Reusability means using existing code to build new software instead of writing everything from scratch. It is a core benefit of modular engineering that saves time and reduces bugs.

How Reusability Works

Write Once: You create a piece of code to solve a specific problem.
Isolate: You package that code so it does not rely on global system settings.
Reuse: You drop that package into other parts of the system or entirely new projects.

Key Benefits

Speed: Faster development since you do not recreate common tools.
Reliability: Code that is reused often has already been tested and debugged.
Consistency: Keeps behavior identical across different screens or apps.

Encapsulation:

Encapsulation means bundling data and the methods that operate on that data into a single unit (like a class) while hiding the internal details from the outside world. It acts as a protective shield that prevents external code from modifying an object's internal state arbitrarily.

Core Concepts

Data Bundling: Variables and functions are grouped together under one roof.
Information Hiding: Internal mechanisms are kept secret from other parts of the application.
Controlled Access: Outside code can only interact with the data through strict public methods.

Key Benefits

Security: Protects your data from accidental corruption by unauthorized external code.
Flexibility: You can change the internal data structure without breaking the apps using it.
Maintainability: Bugs are easier to isolate because the state changes happen in one clear place.

Abstraction:

Abstraction **hides complex background implementation details** and exposes only the **essential features** to the **user**. It helps to reduce programming complexity and allows developers to change internal implementation details without impacting external users.

How it works: Developers create simple interfaces or abstract classes to interact with complex internal logic.

Real-world analogy: When you drive a car, you push the gas pedal to speed up. You do not need to understand how the engine burns fuel or manages the transmission to use it.

Inheritance:

Inheritance is a mechanism that allows a new class (child class) to **derive properties and behaviors** from an existing class (parent class). This promotes code reusability and establishes a logical hierarchy.

How it works: The child class automatically acquires the methods and variables of the parent class, but can also define its own unique attributes.

Real-world analogy: A "Car" class and a "Truck" class can both inherit common traits like "wheels" and "engine" from a base "Vehicle" parent class.

Polymorphism:

Polymorphism translates to "many forms". It allows different classes to be treated as instances of the same parent class, enabling a single interface to **execute different actions depending on the object** invoking it.

How it works: It is achieved via method overloading (defining multiple methods with the same name but different parameters) and method overriding (redefining a parent method inside a child class).

Real-world analogy: Think of a single "Play" command. If you issue it to an audio player, it streams music; if you issue it to a theatre actor, they begin a stage performance.

Better code organization:

Better code organization in OOP means structuring a program into well-designed classes and objects, where each class has a single responsibility and uses OOP principles such as encapsulation, inheritance, polymorphism, and abstraction to make the code modular, reusable, maintainable, and easy to understand.

Benefits of Better Code Organization in OOP

- Easier to read and understand.
- Easier to maintain and debug.
- Promotes code reusability.
- Reduces code duplication.
- Makes it easier to add new features without affecting existing code.
- Improves teamwork because responsibilities are clearly divided.

Maintenance:

Maintenance in OOP is the process of modifying and improving software after development. OOP makes maintenance easier by organizing code into independent, reusable classes using encapsulation, inheritance, polymorphism, and abstraction, allowing changes with minimal impact on the rest of the program.

Benefits of Maintainable OOP code

- Easy to fix bugs.
- Easy to add new features.
- Less code duplication.
- Easier to understand and modify.
- Changes in one class have minimal impact on other classes.

Collaborative Development:

Collaborative development in OOPS is a software development approach where multiple programmers work together on the same project by dividing it into independent classes, objects, or modules. OOP principles make teamwork easier because each developer can work on different parts of the application with minimal interference.

Benefits of Collaborative Development

- Improves teamwork and productivity.
- Reduces code duplication.
- Makes software easier to maintain and update.
- Simplifies testing and debugging.
- Enables faster development of large applications.

Code Reuse:

Code reusability in **Object-Oriented Programming (OOP)** is the ability to use existing code (classes, methods, or objects) in new programs or different parts of the same program without rewriting it. It is one of the main advantages of OOP and helps reduce development time and effort.

Benefits of Code Reuse

- Reduces code duplication.
- Saves development time and effort.
- Improves software quality.
- Makes programs easier to maintain and update.
- Increases productivity.
- Reduces the chances of errors.

Scalability:

Scalability in **Object-Oriented Programming (OOP)** is the ability of a software system to **grow and handle increased requirements** by adding new features, classes, or modules without making major changes to the existing code.

Benefits of scalability

- Easy to add new features.
- Reduces the need to rewrite existing code.
- Simplifies maintenance and upgrades.
- Supports the development of large and complex applications.
- Improves software flexibility and longevity.

4. Applications of OOPS:

Object-Oriented Programming (OOP) is widely used in software development because it provides modularity, reusability, security, and scalability.

- **Desktop Applications**
  Used to develop applications such as text editors, media players, and office software.
  Example: Microsoft Office, Adobe Photoshop.

- **Web Applications**
  Used to build dynamic websites and web-based systems.
  Example: Online banking, e-commerce websites.

- **Mobile Applications**
  Used for Android and other mobile app development.
  Example: WhatsApp, Google Maps.

- **Game Development**
  OOP helps create game objects such as players, enemies, and weapons.
  Example: Unity game engine, Unreal Engine.

- **Banking Systems**
  Used to manage customer accounts, transactions, loans, and ATM operations.

- **Hospital Management Systems**
  Manages patient records, appointments, billing, and medical reports.

- **Library Management Systems**
  Handles book records, member details, book issue, and return operations.

- **E-commerce Applications**
  Used for online shopping, product management, payment processing, and order tracking.
  Example: Amazon, Flip kart.

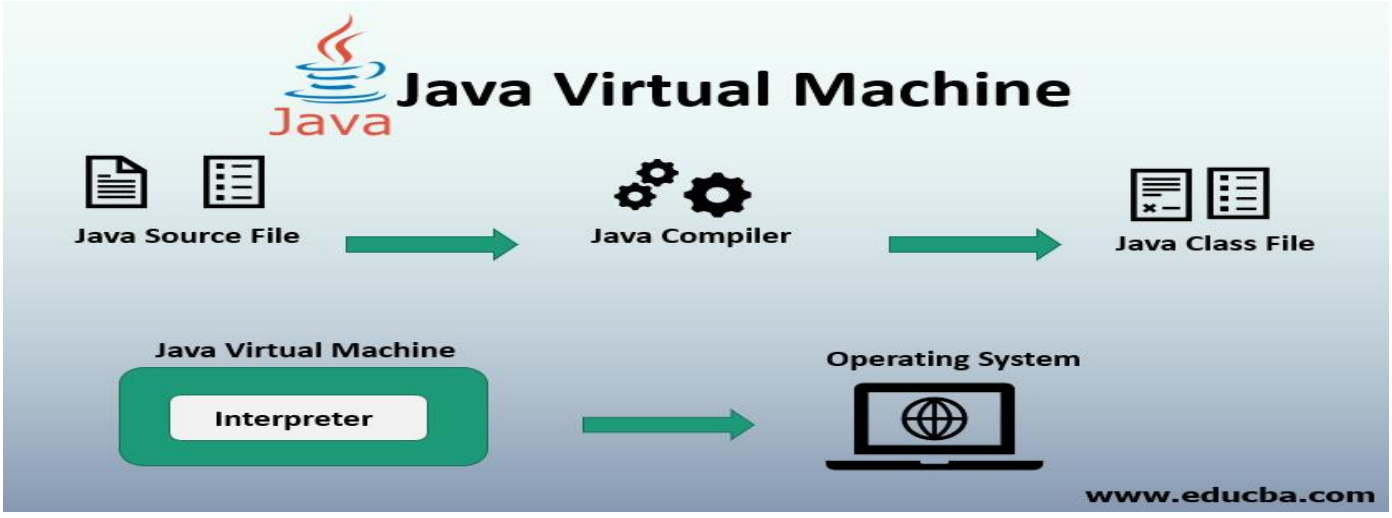
- **Real-Time Systems**
  Used in air traffic control, railway reservation, and traffic management systems.

- **Enterprise Applications**
  Used to develop large business applications such as ERP and CRM systems.

5. JVM (Java Virtual Machine):

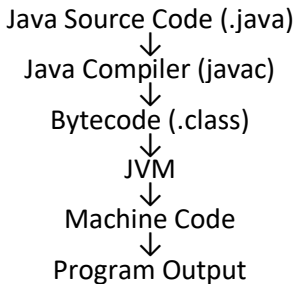
JVM (Java Virtual Machine) is a part of the Java Runtime Environment (JRE) that executes Java **bytecode**. It acts as an intermediary between Java programs and the operating system, allowing Java programs to run on any platform.

JVM is a virtual machine that **converts Java bytecode** into **machine code** and executes it on the computer.



How JVM Works

- Java source code (`.java`) is written by the programmer.
- The Java compiler (`javac`) compiles the source code into **bytecode** (`.class` file).
- The JVM loads the bytecode.
- The JVM converts the bytecode into machine code using the **Just-In-Time (JIT) Compiler**.
- The machine code is executed by the computer.



Functions of JVM

- Loads Java class files.
- Verifies the bytecode for security.
- Converts bytecode into machine code.
- Executes Java programs.
- Manages memory using **Garbage Collection**.
- Provides platform independence ("Write Once, Run Anywhere").

Advantages of JVM

- Platform independent.
- Automatic memory management.
- Secure execution environment.
- Better performance with the JIT compiler.
- Supports multithreading.

JDK Tools:

- **`javac`:** The Java Compiler that reads source code (`.java`) and converts it into bytecode (`.class`).
- **`java`:** The Application Launcher that boots the JVM and runs your compiled class files or packaged JARs.
- **`javap`:** The Class File Dis-assembler used to inspect bytecode instructions inside a compiled class.
- **`javadoc`:** The Documentation Generator that parses code comments into structured HTML API pages.
- **`jdb`:** The built-in command-line Java Debugger used to step through code, set breakpoints, and find bugs.
- **`jar`:** The Archiver used to package multiple class files and resources into a single compressed archive file.
- **`appletviewer:`** This enablesus to view and run java applet programs

Java Fundamentals:

History of Java

- Java programming language was originally developed by **Sun Microsystems** which was initiated by **James Gosling** and **released** in **1995** as core component of **Sun Microsystems' Java platform** (Java 1.0 [J2SE]). History of even naming of the Java is very interesting. It went under many names.

- **James Gosling** was leading a team named as **'Green' team**. Target of this team was to create a new language which can work on multiple **Consumer Electronic Devices** seamlessly. Initially **C++** was the original choice to develop the project. **James Gosling** wanted to enhance **C++** to achieve the target but due to **high memory usage**, that idea was rejected and team started with a new language initially named as **Green Talk**. The file extension used as **.gt**. Later this language was termed as **Oak** and finally to **Java**.

- **James Gosling renamed** language to **Oak**. There was an **Oak tree** in front of his office. James Gosling used this name as Oak represents **solidarity** and **Oak tree** is the **national tree** of multiple countries like **USA**, **France**, **and Romania** etc. But Oak technologies already had Oak as a trademark and James team had to **brainstorm** another title for the language.

- Team put multiple names like **DNA**, **Silk**, **Ruby** and **Java**. Java was finalized by the team. James Gosling tabled Java title based on type of espresso coffee bean. **Java** is an **island** in **Indonesia** where new coffee was discovered termed as Java coffee.

- The team **developed** a web browser called "**HotJava**" in 1994 to locate and run applet programs on the Internet, thus making this language **instantly popular** among the **internet users**.

- Sun released the first public implementation as **Java 1.0** in **1995**. It promised **Write Once Run Anywhere** (**WORA**), providing **no-cost run-times** on popular platforms.

- On **13 November**, **2006**, Sun released much of Java as **free** and **open source software** under the terms of the **GNU** General Public License (**GPL**).

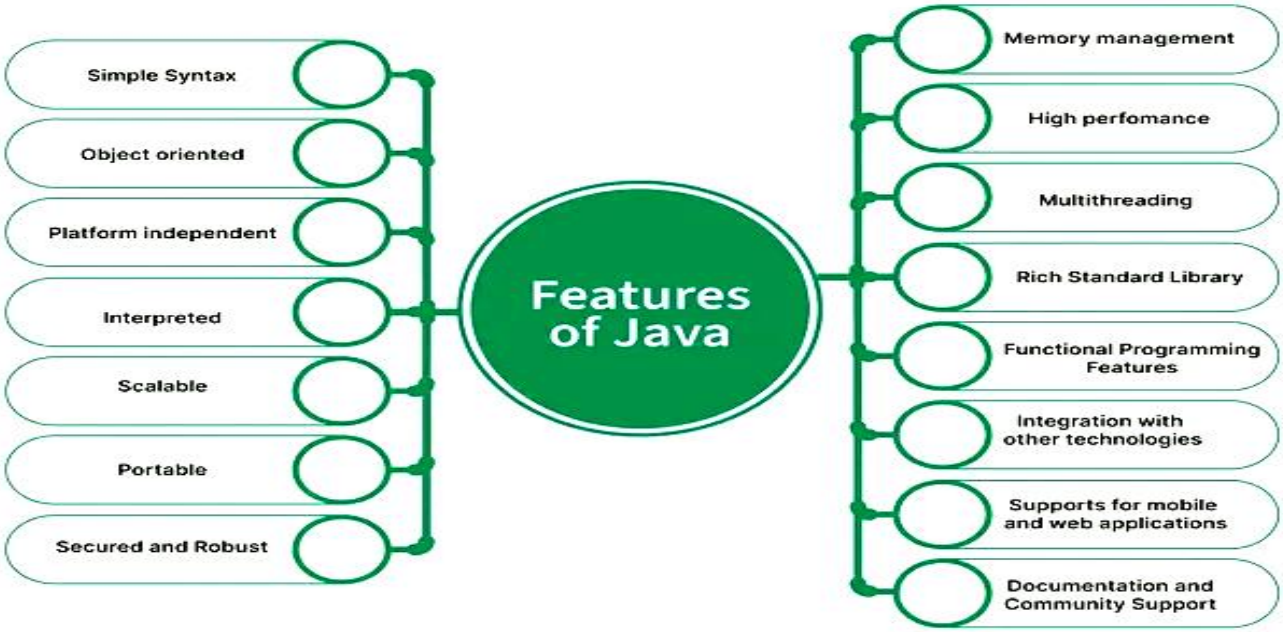
- On **8 May**, **2007**, Sun finished the **process**, making all of Java's core code **free** and **open-source**, aside from a small portion of code to which Sun did not hold the copyright.

- The latest release of the **Java Standard Edition** is **Java SE 21**. With the advancement of Java and its widespread popularity, multiple configurations were built to suit various types of platforms. **For Example**: **J2EE** for Enterprise Applications, **J2ME** for Mobile Applications.

- The various versions of Java are JDK Beta, JDK 1.0, JDK 1.1, JDK 1.2, JDK 1.3, JDK 1.4, JDK 1.5 OR J2SE 5, and JAVA SE 6 to 23. The **recent** and **latest** version of Java is **JAVA SE 23**.

Features of Java:

Java is a High-Level, **object-oriented** programming language. This language is very easy to learn and widely used. It is known for its platform independence, reliability, and security. It follows one principle that is "**Write Once, Run Anywhere**" principle. It supports various features like portability, robustness, simplicity, multithreading, and high performance, which makes it a popular choice for beginners as well as for developers.



1. Simple Syntax:

Java syntax is very straightforward and very easy to learn. Java removes complex features like pointers and multiple inheritance, which makes it a good choice for beginners.

```
import java.io.*;

class Sample
{
        public static void main(String args[])
        {
                System.out.println("Welcome to the World of Java!");
        }
}
```

In **Java**, the execution starts with the **main method**, which is the **entry point** of any **Java application**. The **System.out.println** statement prints "Welcome to the World of Java!". The **import java.io.*;** statement means we are putting input-output functionalities.

2. Object Oriented:

Java is a pure object-oriented language. It supports core OOP concepts like, Class, Object, Encapsulation, Polymorphism, Inheritance and Data Abstraction.

```
Import java.io.*;
class Student
 {
        int age;
        String name;
       public Student(int age, String name)
        {
                    this.age = age;
                    this.name = name;
         }

         void display()
         {
                    System.out.println("Name is: " + name);
                    System.out.println("Age is: " + age);
         }
}
class Sample
 {
       public static void main(String args[])
       {
         Student S = new Student(14, "Akshara");
         S.display();
       }
}
```

Output

```
Name is: Akshara
Age is:  14
```

In the above example, we have created a Student class and inside the class we have declared two variables age and name. A constructor is used to initialize these variables when an object of the Student class is created. In the main method we are creating an object of the student class and then we are calling the display method which prints the name and age on the console.

3. Platform Independent:

Java is **platform-independent** because of **Java Virtual Machine (JVM).**

- When we write Java code, it is first compiled by the compiler and then converted into bytecode (which is platform-independent).
- This byte code can run on any platform which has JVM installed.

4. Interpreted:

Java code is not directly executed by the computer. It is first compiled into bytecode. This byte code is then understood by the JVM. This enables Java to run on any platform without rewriting code.

5. Scalable:

Java can handle both small and large-scale applications. Java provides features like multithreading and distributed computing that allows developers to manage loads more easily.

6. Portable:

When we write a Java program, the code first gets converted into bytecode and this bytecode does not depend on any operating system or any specific computer. We can simply execute this bytecode on any platform with the help of JVM. Since JVM is available on most devices and that's why we can run the same Java program on different platform.

7. Secure and Robust:

Java is a reliable programming language because it can catch mistakes early while writing the code and also keeps checking for errors when the program is running. It also has a feature called exception handling that helps deal with unexpected problems smoothly.

8. Memory Management:

Memory management in Java is automatically handled by the **Java Virtual Machine (JVM)**.

- Java **garbage collector** reclaim memory from objects that are no longer needed.
- Memory for objects are allocated in the heap
- Method calls and local variables are stored in the stack.

9. High Performance:

Java is faster than old interpreted languages. Java program is first converted into bytecode which is faster than interpreted code. It is slower than fully compiled languages like C or C++ because of interpretation and JIT compilation process. Java performance is improve with the help of Just-In-Time (JIT) compilation, which makes it faster than many interpreted languages but not as fast as fully compiled languages.

10. Multithreading:

Multithreading in Java allows multiple threads to run at the same time.

- It improves CPU utilization and enhancing performance in applications that require concurrent task execution.
- Multithreading is especially important for interactive and high-performance applications, such as games and real-time systems.
- Java provides build in support for managing multiple threads. A **thread** is known as the smallest unit of execution within a process.

11. Rich Standard Library:

Java provides various pre-built tools and libraries which is known as Java API. Java API is used to cover tasks like file handling, networking, database connectivity (JDBC), security, etc. With the help of these libraries developers save a lot of time and ready to use solutions and can also build a powerful application.

12. Functional Programming Features:

Since Java 8, the language has introduced functional programming features such as:

- Lambda expression let us to write small block of code in a very easy way without creating full methods.
- Stream API allows data to be processed easily, Instead of writing long loops we can just filter, change, or combine data in a few lines.
- Functional interfaces are interface that contains only one method. They work perfectly with lambda expressions and help us write flexible and reusable code.

13. Integration with other Technologies:

Java can easily work with many languages and tools as well. For example, Java can connect with C and C++ with the help of **Java Native Interface (JNI)**. Java is very popular for building **websites and web services** like **RESTful & SOAP**. In Java we can use **JDBC for database connectivity** also Java is the main language for android development. As we can see Java works so well with so many different technologies that's the reason developer prefers Java more to create scalable and powerful application.

14. Support for Mobile and Web Applications:

Java offers support for both web and mobile applications.

- For **web development:** Java offers technologies like JSP and Servlets, along with frameworks like Spring and Spring boot, which makes it easier to build web applications.
- For **mobile development:** Java is the main language for Android app development. The Android SDK uses special version of Java and its various tools to build mobile apps for Android devices.

15. Documentation and Community Support:

Java provides **documentation** which includes guides, API references, and tutorials for easy learning. Java has a **large** and **active global community** contributing to open-source projects, and resources. This community support helps developers **solve problems** and stay updated with new advancements.

Variables in Java:

A **variable** in Java is a named memory location used to store data. The value stored in a variable can change during program execution. Every variable has a **data type**, a **name (identifier)**, and optionally an **initial value**.

Syntax: DataType Variable_Name = Value;

Example:

```
int age = 20;
String Name = "Akshara";
double Salary = 45000.50;
 char Grade = 'A';
```

Types of Variables in Java:

Local Variable

- Declared inside a **Method**, **Constructor**, or **Block**.
- Accessible only within that Block.

Instance Variable

- Declared inside a **Class** but **outside Method**.
- Each object has its own copy.

Static Variable (Class Variable)

- Declared with the **static keyword**.
- Shared by all **Objects** of the **Class**.

Rules for Naming Variables

- Must start with a letter, _, or $.
- Cannot start with a number.
- Cannot use Java keywords (e.g., int, class, if, else, for…).
- Variable Names are **case-sensitive** (age and Age are different).
- Use meaningful names, such as Student_Name or Total_Marks.

```
public class Sample
{
        public static void main(String[] args)
        {
              int Age = 21;
              String Name = "Alice";
              double Height = 5.6;
              boolean isStudent = true;
              System.out.println("Name: " + Name);
              System.out.println("Age: " + Age);
              System.out.println("Height: " + Height);
              System.out.println("Student: " + isStudent);
        }
}
```

Constants in Java:

In Java, a **constant** is a variable whose value **cannot be changed** after it is assigned. Constants are created using the `final` keyword.

Syntax: **final DataType Constant_Name = Value;**

```
public class Sample
{
	Public static void main(String args[])
	{
		final double PI = 3.14159;
		System.out.prinln(PI);
	}
}
```

When a constant belongs to the entire class (not individual objects), use `static final`.

```
public class Sample
{
	public static final double PI = 3.14159;
	public static final int MAX_USERS = 100;

	Public static void main(String args[])
	{
		final double PI = 3.14159;
		System.out.prinln(PI);
	}
}
```

Key Points by Java convention:

- Write **constant names** in **UPPERCASE**.
- Separate multiple words with **underscores (_).**
- **final** means the variable can be assigned **only once**.
- **static final** is used for **class-level constants** shared by all objects.
- **final** reference variable **cannot point** to a different **object**, but the object's internal state may still change if it is mutable.

```
public class Sample
{
	public static final double PI = 3.14159;

	Public static void main(String args[])
	{
		double radius = 5;
		double Area = PI * radius * radius;
		System.out.prinln("Area of Circle =" + Area);
	}
}
```

Data types in Java:

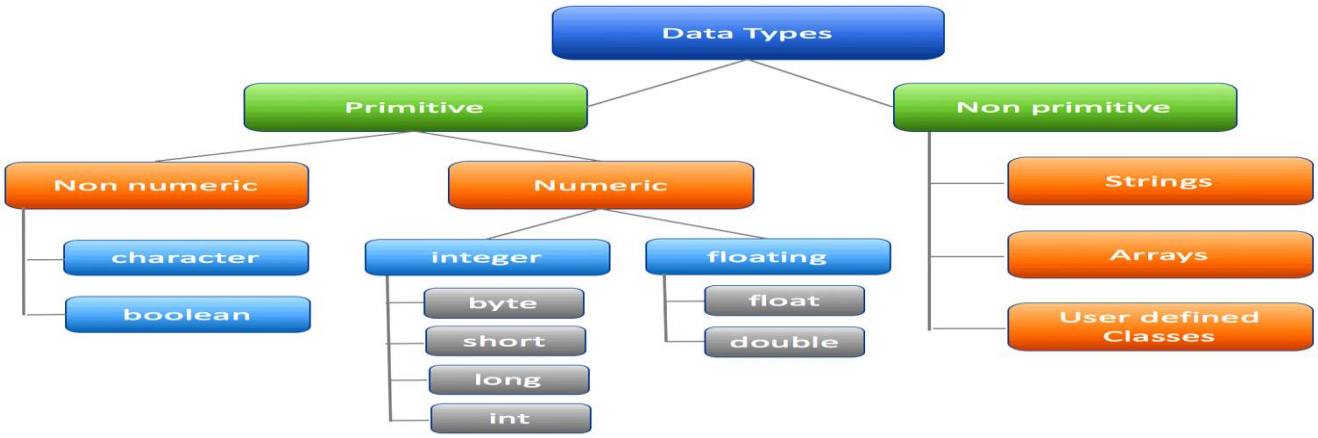
Java data types define the type of data a variable can store in a program. They help the compiler allocate memory efficiently and ensure type safety.

- Memory allocation determines how much memory is required for each variable.
- Operations support defines what operations can be performed on data.
- Each data type has a default value when not initialized.

```
DataType variable_Name = value;
```

Java data types broadly divided into two categories:

- **Primitive Data Types:** Store simple values directly in memory.
- **Non-Primitive (Reference) Data Types:** Store memory references to objects.



Primitive Data Types:

Primitive data types store simple values directly in memory. Java provides eight primitive data types, each with a fixed size and range.

Boolean Data Type:

Represents one of two logical values: true or false. It is commonly used in conditions and control statements.

Syntax: `boolean Boolean_Var_name;`

```
public class Sample
{

	public static void main(String[] args)
	{
		boolean isJavaFun = true;
		boolean isFishTasty = false;
		System.out.println("Is Java fun? " + isJavaFun);
		System.out.println("Is fish tasty? " + isFishTasty);
	}
}
```

Byte Data Type: An 8-bit signed integer used to save memory in large numeric arrays.

Syntax: `byte byte_Var_Name;`

```
public class Sample
{
    public static void main(String[] args)
    {
          byte Age = 25;
          byte Temp = -10;
          System.out.println("Age: " + age);
          System.out.println("Temperature: " + Temperature);
    }
}
```

Short Data Type:

A 16-bit signed integer often used when memory is limited and values are moderate in size.

Syntax: short short_Var_Name;

```
public class Sample
{
     public static void main(String[] args)
    {
              short Students = 1000;
              short temp = -200;
              System.out.println("Number's of Students: " + students);
              System.out.println("Temperature: " + temp);
    }
}
```

Int Data Type: A 32-bit signed integer and the most commonly used numeric data type.

Syntax: `int int_Var_Name;`

```
public class Sample
{
     public static void main(String[] args)
    {
              int population = 2000000;
              int distance = 150000000;
              System.out.println("Population: " + population);
              System.out.println("Distance: " + distance);
     }
}
```

Long Data Type: A 64-bit signed integer used when int is not sufficient for large values.

Syntax: `long long_Var_Name;`

```
public class Sample
{
	public static void main(String[] args)
	{
		long worldPopulation = 7800000000L;
		long lightYears = 9460730472580800L;
		System.out.println("World Population: " + worldPopulation);
		System.out.println("Light Years: " + lightYears);
	}
}
```

Float Data Type: A 32-bit single-precision floating-point type used for fractional values.

Syntax: `float float_Var_Name;`

```
public class Sample
{
	public static void main(String[] args)
	{
		float pi = 3.14f;
		float gravity = 9.81f;
		System.out.println("Pi: " + pi);
		System.out.println("Gravity: " + gravity);
	}
}
```

Double Data Type: A 64-bit double-precision floating-point type and the default for decimal numbers.

Syntax: `double double_Var_Name;`

```
public class Sample
{
	public static void main(String[] args)
	{
		double pi = 3.141592653589793;
		double avogadro = 6.02214076e23;
		System.out.println("Pi: " + pi);
		System.out.println("Avogadro's Number: " + avogadro);
	}
}
```

Char Data Type: A 16-bit Unicode character used to store single symbols or letters.

Syntax: `char char_Var_Name;`

```
public class Sample
{
	public static void main(String[] args)
	{
		char grade = 'A';
		char symbol = '$';
		System.out.println("Grade: " + grade);
		System.out.println("Symbol: " + symbol);
	}
}
```

Non-Primitive (Reference) Data Types:

Non-primitive data types store references (memory addresses) rather than actual values. They are created by users and include types like String, Class, Object, Interface, and Array.

String: String represents a sequence of characters enclosed in double quotes. Unlike C/C++, Java strings are Objects and are immutable.

Syntax: `String Str_Var = "Hello";`

```
public class Sample
{
	public static void main(String[] args)
		{
			String name = "Geek1";
			String message = "Welcome to Java";
			System.out.println("Name: " + name);
			System.out.println("Message: " + message);
		}
}
```

Class: Class is a user-defined dataType that defines variables and methods. It represents a type of object and forms the foundation of Object-Oriented Programming.

```
class Car
{
	String model;
	int year;

	Car(String model, int year)
	{
		this.model = model;
		this.year = year;
	}

	void display()
	{
		System.out.println(model + " " + year);
	}
}

public class Sample
{
	public static void main(String[] args)
	{
		Car myCar = new Car("Toyota", 2020);
		myCar.display();
	}
}
```

Object: An Object is an instance of a class representing real-world entities. It has state (data), behavior (methods), and identity (unique reference).

```
class Car
{
	String model;
	int year;

	Car(String model, int year)
	{
		this.model = model;	this.year = year;
	}
}

public class Sample
{
	public static void main(String[] args)
	{
		Car myCar = new Car("Honda", 2021);
		System.out.println("Model: " + myCar.model);
		System.out.println("Year: " + myCar.year);
	}
}
```

Interface: An Interface defines a contract of abstract methods that implementing classes must define. It provides a way to achieve abstraction and multiple inheritances in Java.

```
interface Animal
{
	void sound();
}

class Cat implements Animal
{
	public void sound()
	{
		System.out.println("Woof");
	}
 }

public class Sample
{
	public static void main(String[] args)
	{
		Animal dog = new Cat();
		dog.sound();
	}
}
```

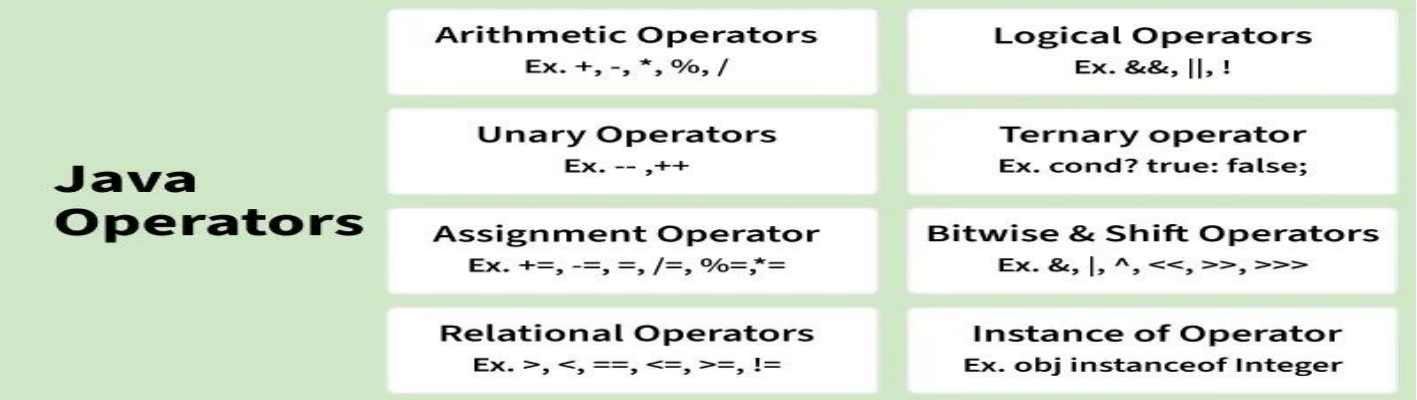
Array: An array stores multiple elements of the same type in a single structure. Java arrays are objects, dynamically allocated, and indexed from 0.

```
public class Sample
{
	public static void main(String[] args)
	{
		int[] Numbers = {1, 2, 3, 4, 5};
		String[] Names = {"One", "Two", "Three"};
		System.out.println("First number: " + Numbers[0]);
		System.out.println("Second name: " + Names[1]);
	}
}
```

Operators in Java:

Java operators are symbols used to perform operations on variables and values. They play a key role in expressions, calculations, and decision-making in programs. Operators help simplify complex logic into concise statements.

- They follow a defined precedence and associativity to determine execution order.
- Some operators work on a single operand (unary), while others require two or more operands.



Arithmetic Operators: Arithmetic Operators are used to perform arithmetic operations on primitive numeric Data types such as int, float, and double.

```
public class Arithmetic
{
    public static void main(String[] args)
 {
        int a = 10, b = 3;
        int sum = a + b;
        int diff = a - b;
        int mul = a * b;
        int div = a / b;
        int mod = a % b;
        System.out.println("Sum: " + sum);
        System.out.println("Difference: " + diff);
        System.out.println("Multiplication: " + mul);
        System.out.println("Division: " + div);
        System.out.println("Modulus: " + mod);
    }
}
```

Output:

```
Sum:   13
Difference:  7
Multiplication:    30
Division:    3
Modulus:     1
```

Unary Operators: These need only one operand. They are used to increment, decrement, or negate a value.

```
import java.io.*;

      class Sample
      {
            public static void main(String[] args)
            {

                  int a = 10;
                  int b = 10;

                  System.out.println("Post increment : " + (a++));
                   System.out.println("Pre increment : " + (++a));

                   System.out.println("Post decrement : " + (b--));
                  System.out.println("Pre decrement : " + (--b));
    }
}
```

Output:

```
Post increment:     10
Pre increment:      11
Post decrement:     10
Pre decrement:      9
```

Assignment Operators: The assignment operator assigns a value from the right-hand side to a variable on the left. Since it has right-to-left associativity, the right-hand value must be declared or constant.

```
public class Assignment
{
    public static void main(String[] args)
    {

                        int num = 10;
                        System.out.println("Initial: " + num);
                        num += 5;
                        System.out.println("After +5: " + num);
                        num *= 2;
                        System.out.println("After *2: " + num);
                        num -= 5;
                        System.out.println("After -5: " + num);
                        num /= 2;
                        System.out.println("After /2: " + num);
                        num %= 3;
                        System.out.println("After %3: " + num);
    }
}
```

Output:

```
Initial:     10
After +5:    15
After *2:    30
After -5:    25
After /2:    12
After %3:    0
```

Relational Operators: Relational Operators are used to check for relations like equality, greater than, and less than. They return boolean results after the comparison and are extensively used in looping statements as well as conditional if-else statements.

```
import java.io.*;

class Relational
{
        public static void main(String[] args)
       {

                        int a = 10;
                        int b = 3;
                        int c = 5;
                        System.out.println("a > b: " + (a > b));
                        System.out.println("a < b: " + (a < b));
                        System.out.println("a >= b: " + (a >= b));
                        System.out.println("a <= b: " + (a <= b));
                        System.out.println("a == c: " + (a == c));
                        System.out.println("a != c: " + (a != c));
       }
}
```

Output:

```
a > b:      true
a < b:      false
a >= b:     true
a <= b:     false
a == b:     false
a != b:     true
```

Logical Operators: Logical Operators are used to perform "logical AND" and "logical OR" operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

```
import java.io.*;
class Logical
{
            public static void main (String[] args)
        {
                        boolean x = true;
                        boolean y = false;
                        System.out.println("x && y: " + (x && y));
                        System.out.println("x || y: " + (x || y));
                        System.out.println("!x: " + (!x));
        }
}
```

Output:

```
x && y:     false
x || y:     true
!x:         false
```

Ternary Operator: The Ternary Operator is a shorthand version of the if-else statement. It has three operands and hence the name Ternary. The general format is,

```
public class Ternary
{
      public static void main(String[] args)
       {
         int a = 20, b = 10, c = 30, result;

         result = ((a > b) ? (a > c) ? a : c : (b > c) ? b : c);

         System.out.println("Max of three numbers = "+ result);
    }
}
```

Output:

```
Max of three numbers =    30
```

Bitwise Operators: These operators perform operations at the bit level.
Bitwise Operators manipulate individual bits using AND, OR, XOR, and NOT.
Shift Operators move bits left or right, effectively multiplying or dividing by powers of two.

```
import java.io.*;
class Bitwise
{
    public static void main(String[] args)
    {
        int d = 0b1010;
        int e = 0b1100;
        System.out.println("d & e : " + (d & e));
        System.out.println("d | e : " + (d | e));
        System.out.println("d ^ e : " + (d ^ e));
        System.out.println("~d : " + (~d));
        System.out.println("d << 2 : " + (d << 2));
        System.out.println("e >> 1 : " + (e >> 1));
        System.out.println("e >>> 1 : " + (e >>> 1));
    }
}
```

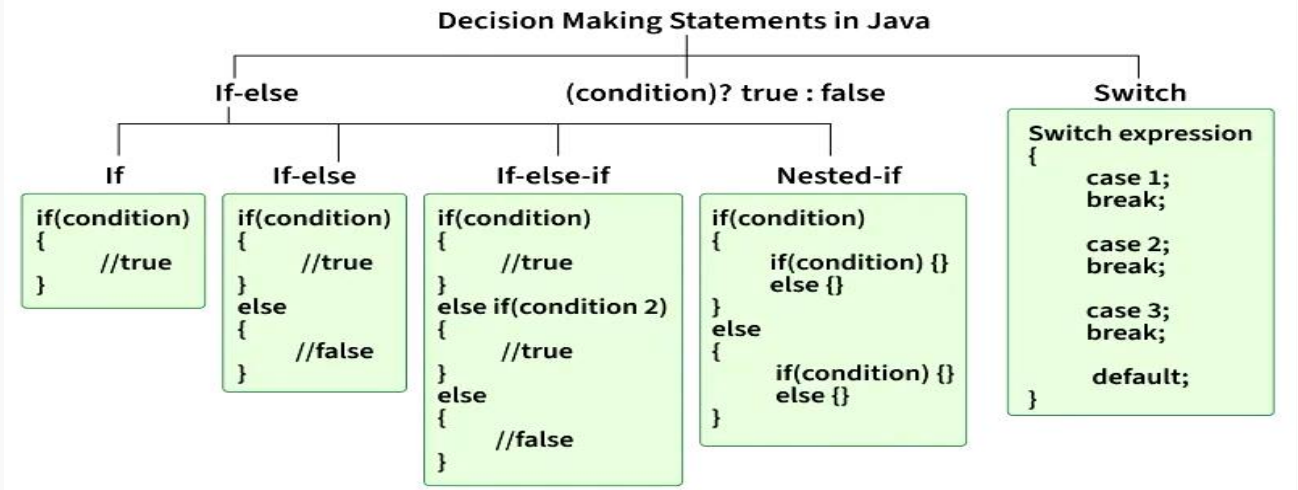
instanceof Operator: The instanceof operator is used for type checking. It can be used to test if an object is an instance of a class, a subclass, or an interface. The general format,

```
public class Sample
 {
    public static void main(String[] args)
    {
        String str = "Hello";
        System.out.println(str instanceof String);
        Object obj = 10;
        System.out.println(obj instanceof Integer);
        System.out.println(obj instanceof String);
    }
}
```

Decision Making and Branching in Java:

These statements in Java allow a program to choose different execution paths based on specified conditions. They help control the flow of a program by executing certain blocks of code only when particular conditions are true. These statements are essential for implementing logic, validations, and user-driven actions in applications.

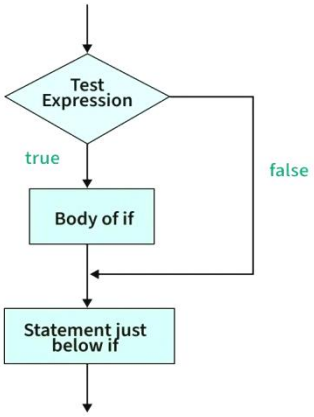
- Improve flexibility and intelligence of programs.
- Support simple, multiple, and nested decision-making scenarios.



If Statement: The **if statement** is the **simplest** and **one way decision-making** statement. It executes a block of code only if a given condition is true.

```
class Sample
{
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
        {
            System.out.println("Condition is True");
        }
    }
}
```



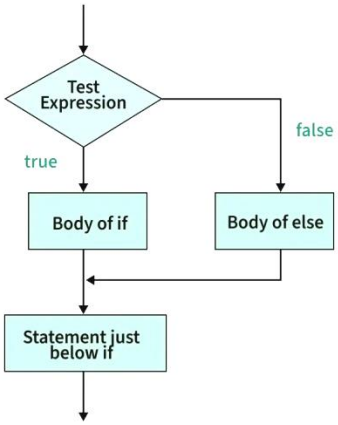
Execution Flow:

- Condition is evaluated.
- If the condition is true, the if block executes.
- If the condition is false, the block is skipped.
- Program continues with the next statement.

If-else Statement: The **if-else** statement allows you to **execute one block** if the condition is **true** and **another block** if it is **false**.

```
import java.util.*;
class Sample
 {
    public static void main(String args[])
    {
        int i = 10;

        if (i < 15)
            System.out.println("i is Smaller than 15");
        else
            System.out.println("i is Greater than 15");
    }
}
```



Execution Flow

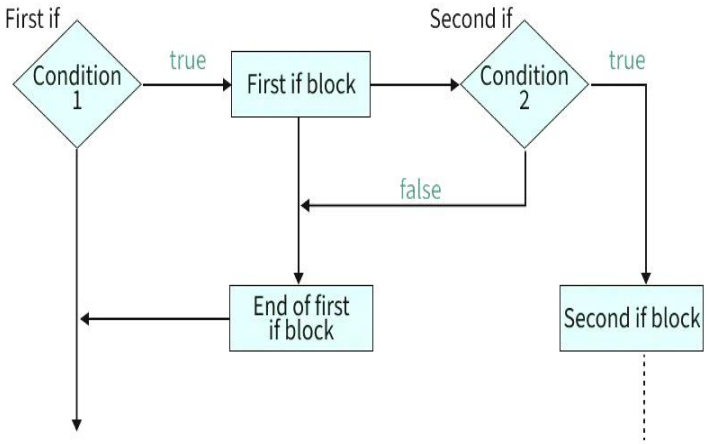
- Condition is evaluated.
- If true, the if block executes.
- If false, the else block executes.
- Program proceeds to the next statement.

Nested-If Statement: The **Nested-if** is an **if statement inside another if statement**. It is useful when a second condition depends on the first.

```
class Sample
{
 public static void main(String args[])
 {
   int i = 10;

   if (i < 15)
   {
    System.out.println("i is smaller than 15");

    if (i == 10)
       {
         System.out.println("i is exactly 10");
       }
   }
 }
}
```



Execution Flow

- Outer if condition is evaluated first.
- If the outer condition is true, the inner if condition is checked.
- If the inner condition is also true, its block executes.
- Program continues after the nested structure.

If-else-if Statement: The **if-else-if** ladder allows **multiple independent conditions** to be checked in order.

As soon as one condition is true, its block executes, and the rest are skipped.

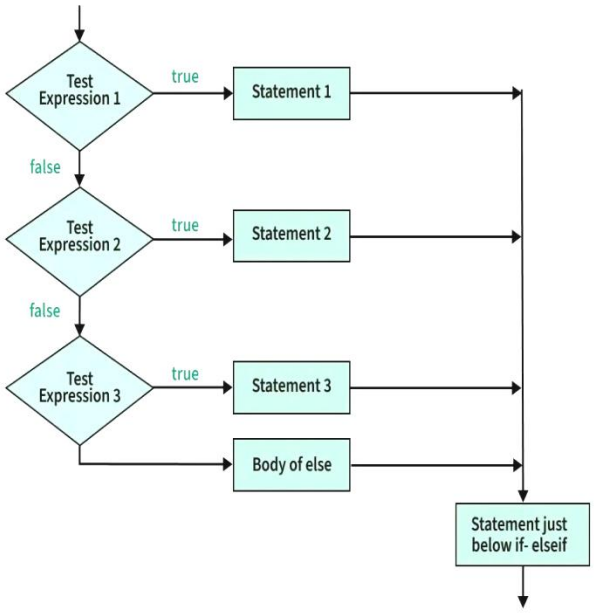
```
import java.util.*;
class Sample
 {
    public static void main(String args[])
    {
        int i = 20;

        if (i == 10)
            System.out.println("i is 10");

        else if (i == 15)
            System.out.println("i is 15");

        else if (i == 20)
            System.out.println("i is 20");

        else
            System.out.println("i is not present");
    }
}
```

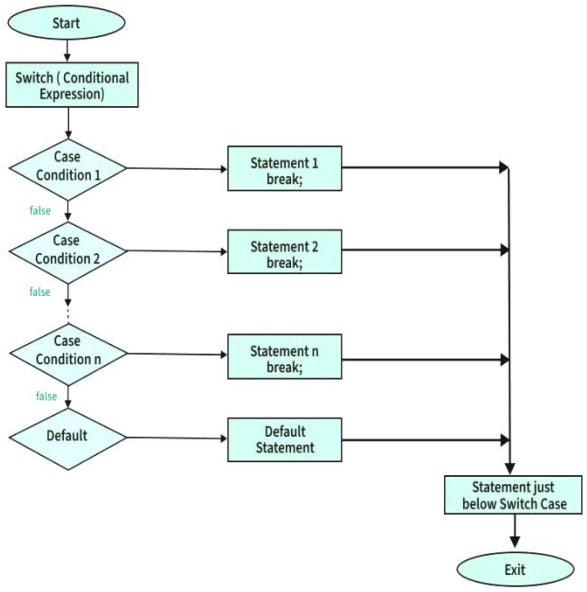


Execution Flow

- Conditions are evaluated from top to bottom.
- The first condition that evaluates to true executes its block.
- Remaining conditions are skipped.
- If no condition is true, the else block executes.

Switch Case Statement: The **switch** statement is a **multi way branch** statement. It provides an easy way to **dispatch execution** to **different parts** of code based on the **value** of the **expression**.

```
class Sample
 {
    public static void main(String[] args)
    {
        int num = 20;
        switch (num)
        {
        case 5:
            System.out.println("It is 5");  break;
        case 10:
            System.out.println("It is 10"); break;
        case 15:
            System.out.println("It is 15"); break;
        case 20:
            System.out.println("It is 20"); break;
        default:
            System.out.println("Not present");
        }
    }
}
```



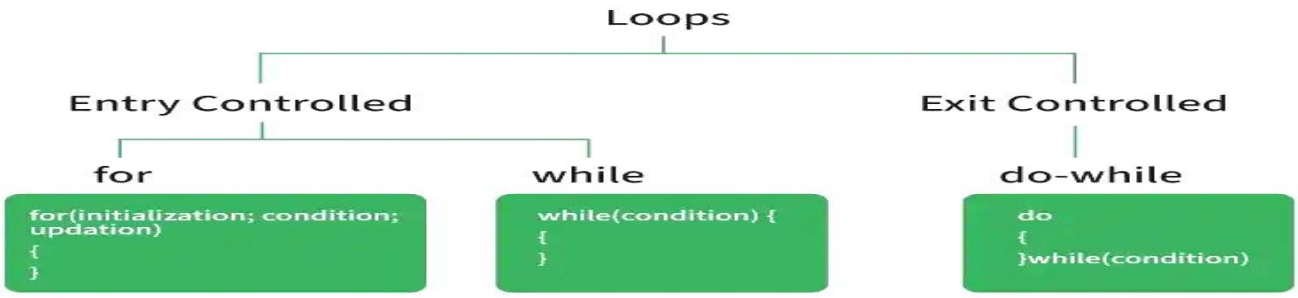
Looping Statements in Java:

Loops in **programming** are used to **execute** a **block** of **code repeatedly until** a **specified condition is met**. They help reduce code duplication and make programs more efficient and readable.

In Java, loops are essential for handling repetitive tasks.

- Allow execution of code multiple times based on a condition
- Improve efficiency by avoiding repetitive code

Java provides different types of loops that allow a block of code to be executed repeatedly based on a specified condition.

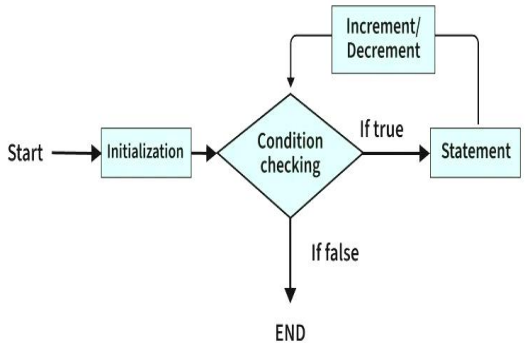


for loop: The **for loop** is used when we know the number of iterations (we know how many times we want to repeat a task). **For statement** includes the **initialization**, **condition**, and **increment/decrement** in one line.

Example: **Program demonstrates the use of** for loop **that prints numbers from 0 to 10 in a single line.**

```
import java.io.*;
class Sample
{
	public static void main(String[] args)
	{
		for (int i = 0; i <= 10; i++)
		{
			System.out.print(i + " ");
		}
	}
}
```

Output: `0 1 2 3 4 5 6 7 8 9 10`

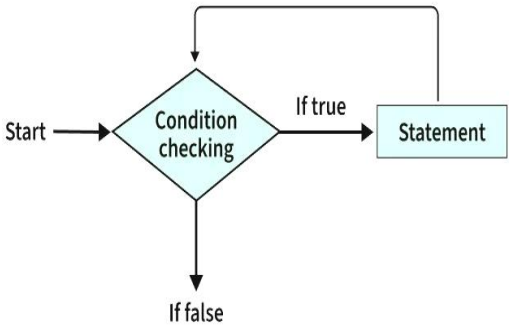


- **Initialization condition**: Here, we initialize the variable in use. It marks the start of for loop. An already declared variable can be used or a variable can be declared, local to loop only.
- **Testing Condition:** It is used for testing the exit condition for a loop. It must return a boolean value. It is also an Entry Control Loop as the condition is checked prior to the execution of the loop statements.
- **Statement execution**: Once the condition is evaluated to true, the statements in the loop body are executed.
- **Increment/ Decrement**: It is used for updating the variable for next iteration.
- **Loop termination**: When the condition becomes false, the loop terminates marking the end of its life cycle.

while Loop: while loop is used when we want to check the condition before executing the loop body.

Example: Program demonstrates a while loop that prints numbers from 0 to 10 in a single line.

```
import java.io.*;
class Sample
 {
    public static void main(String[] args)
    {
        int i = 0;
        while(i <= 10)
        {
            System.out.print(i + " ");
            i++;
        }
    }
}
```



Output: 0 1 2 3 4 5 6 7 8 9 10

- While loop starts with the checking of Boolean condition. If it evaluated to true, then the loop body statements are executed otherwise first statement following the loop is executed. For this reason it is also called Entry control loop.
- Once the condition is evaluated to true, the statements in the loop body are executed. Normally the statements contain an update value for the variable being processed for the next iteration.
- When the condition becomes false, the loop terminates which marks the end of its life cycle.

Arrays in Java:

An array is a ordered collection of elements of the same data type stored in contiguous memory locations. It allows multiple values to be stored under a single name and accessed using an index.

- Java arrays can hold both primitive types (like int, char, boolean, etc.) and objects (like String, Integer, etc.)
- When we use arrays of primitive types, the elements are stored in contiguous locations. For non primitive types, references to items are stored at contiguous locations.
- After creating an array, its size is fixed; we cannot change it.

Declaration of Array: In Java, an array is declared by specifying the data type, followed by the array name, and Empty square brackets [].

Syntax: dataType[] Array_Name; **OR** dataType Array_Name[];

Initialization of Array:

Syntax: int Array[] = new int[size]; **OR** int[] Array = {1, 2, 3};

```
public class Array
{
        public static void main(String[] args)
    {
         int[] arr = {10, 20, 30, 40};   int n = arr.length;
         System.out.print("Primitive Array -> ");
         for (int i = 0; i < n; i++)
            System.out.print(arr[i] + " ");
         System.out.println();
         String[] Names = {"Lakshit", "Rahul", "Pankaj"};
         System.out.print("Non-Primitive Array -> ");
         for (int i = 0; i < names.length; i++)
             System.out.print(names[i] + " ");
    }
}
```

Output:

```
Primitive Array ->  10    20    30    40
Non-Primitive Array ->    Lakshit      Rahul  Pankaj
```

```
class Sample
{
        public static void main(String[] args)
       {
              int[] arr = {2, 4, 8, 12, 16};
              System.out.print(arr[3] + " ");
               System.out.print(arr[0]);
       }
}
```

Output: 12 2

```
class Sample
{
 public static void main(String[] args)
      {
        int[] arr = {2, 4, 8, 12, 16};
        arr[0] = 90;   System.out.println(arr[0]);
      }
}
```

Output: 90

```
class Sample
{
       public static void main(String[] args)
      {
        int[] arr = {2, 4, 8, 12, 16};
        for (int i = 0; i < arr.length; i++)
         {
              System.out.print(arr[i] + " ");
        }
    }
}
```

Output: 2 4 8 12 16